

The Same Class, in Python

Read the main notes first. This sheet does not re-teach the topic. It assumes you already have the Java picture in your head — the stack frame, the heap, the arrow from a reference to an object — and shows you only **what changes when the language is Python**. Every claim here was executed on CPython 3.11; the outputs shown are real.

★ **SAME** — carries over from Java ⚠ **DIFFERENT** — Python does not work like Java here ► **NOTE** — clarification

◆ **INTERVIEW** — where it shows up

0 · The three questions, answered in one table

Your question	Python	The short answer
Do we have the same concept of a call stack?	Yes	Identical. One frame per call, pushed on call, popped on return. You have been looking at it all along — a traceback is the call stack printed out .
Do we have the same concept of heap memory?	Yes — more so	Everything is a heap object. Not "objects go on the heap and primitives don't" — Python has no primitives at all . Even 10 is an object with an address.
Is passing a parameter still by value?	Yes	In exactly the sense of Java's <i>non-primitive</i> case, and always. A reference is copied into the parameter. Python simply never has the primitive case, so there is one rule instead of two.

★ IF YOU REMEMBER ONE THING

Take the Java picture and **delete the green boxes** — the ones where the value sat inside the frame. In Python every box in a frame is a white box holding an address. That single edit converts the class diagram into the Python diagram.

1 · Call stack — unchanged, and visible

nothing new to learn

The three functions from class, in Python:

```
def add(x, y):      return x + y
def product(x, y):  return x * y
def subtract(x, y): return x - y

x, y = 10, 20
t1, t2, t3 = add(x, y), product(x, y), subtract(x, y)
print(t1 + t2 + t3)      # 220
```

★ **SAME**
Output is 220, verified. The call stack concept is identical — Python just lets you look at it while the program is running.

Same frames, same order, same picture. But in Python you can simply ask the interpreter to print the stack:

```
import traceback
def show_stack():
    for f in traceback.extract_stack()[::-1]:
        print(f.name, f.lineno)

def level3(): show_stack()
def level2(): level3()
def level1(): level2()
level1()
```

```
frames currently on the stack, top first:
show_stack()   line 14
level3()       line 18
level2()       line 19
level1()       line 20
<module>()     line 51 ← the bottom frame
```

```
print(sys.getrecursionlimit()) # 1000

def recurse(n): return recurse(n + 1)
recurse(0)
# RecursionError: maximum recursion depth exceeded
```

★ SAME IDEA, FRIENDLIER FAILURE

Java threw `StackOverflowError` when a runaway recursion filled the stack. Python raises `RecursionError` — but it does not wait for the real stack to fill up. It counts frames and stops at a **soft limit of 1000**, so it fails early and tells you clearly.

► THE REALISATION

Read that output bottom-up and it is a stack traceback. **Every traceback you have ever debugged was this diagram**, printed at the moment something threw.

► NOTE

Python has no `main()`. The module itself is the bottom frame, shown as `<module>`. Everything else about the diagram from class is unchanged.

⚠ DIFFERENT — AND IT BITES IN DSA

1000 frames is not many. A recursive DFS on a graph of 10^5 nodes will hit `RecursionError` in Python while the identical Java code runs fine. Either raise the limit with `sys.setrecursionlimit()` or rewrite the recursion as a loop with an explicit stack.

◆ INTERVIEW

"Convert this recursion to iteration" is asked partly because of exactly this. And the tool you use to do it is the stack data structure from section 1 of the class notes — you hand-build the call stack the language was building for you.

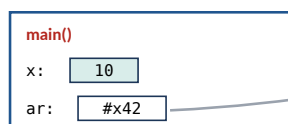
The Java rule was: **primitives and references live in the frame; objects live in the heap**. Python keeps the second half and deletes the first, because there is nothing to put in it.

```
n = 10
type(n)      # int      - it is an object
hex(id(n))   # 0xa2c208 - it has an address
n.bit_length() # 4      - it has methods
```

THE PYTHON RULE

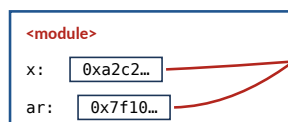
There are no primitives. 10 is a heap object. 3.5 is a heap object. True is a heap object. A name in a frame is **always** a reference — an arrow to something in the heap. Nothing else ever sits in a frame.

JAVA — from class



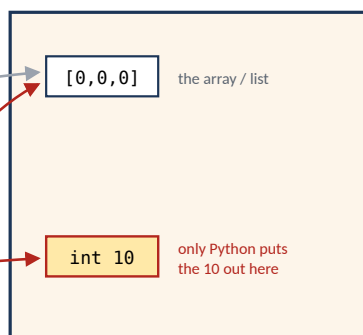
the 10 is IN the frame — a green box

PYTHON — same code



both boxes are white — every name is an arrow

HEAP



★ SAME

The heap works identically: objects outlive the frame, they are reached through an address, and two names holding the same address see the same object.

⚠ DIFFERENT

Java's two-case rule ("is it a primitive or not?") does not exist here. There is one case. Every question of the form "will the caller see the change?" is answered by Java's *non-primitive* reasoning, always.

► NOTE

`id(x)` is Python's version of printing a reference — the thing Java would not let you see. Two names with the same `id` point at the same object, which is what `is` tests.

◆ INTERVIEW

"What is the difference between `==` and `is`?" — `==` compares the contents at the ends of the two arrows; `is` compares the arrows.

Aliasing and garbage collection — the class examples, unchanged

```
ar = [0, 0, 0]
ar2 = ar          # copies the ARROW, not the list
hex(id(ar))       # 0x7f10353fcec0
hex(id(ar2))      # 0x7f10353fcec0 ← identical
ar2 is ar         # True
ar2[0] = 99
ar                # [99, 0, 0] ← Example 2, exactly
```

```
c = [0, 0, 0]
c[1], c[2] = 9, 5 # [0, 9, 5] at 0x7f10353e6240
c = [0] * 5       # [0,0,0,0,0] at 0x7f10353e5180
                  # old list now unreachable — Example 3
```

★ SAME

Examples 2 and 3 from class translate line for line, and the diagrams are the same drawings. Python also frees the abandoned list for you — so the last step of Example 3, the garbage collector, is unchanged too.

► HOW PYTHON'S GC DIFFERS

Java's collector periodically hunts for unreachable objects. Python mainly uses **reference counting**: each object knows how many arrows point at it, and is freed the instant that hits zero. A second collector exists only to catch reference cycles.

► CONSEQUENCE

Because of reference counting, Python frees the old list *immediately* at the moment you rebind, not "eventually". Same outcome, more predictable timing.

3 · How a method call passes its parameters

one rule instead of two

THE RULE

The **reference** in the caller's variable is copied into the parameter. Two names, one object — every single time. This is Java's Case 2, applied to everything.

So why does Case 1 from class still print 10 in Python? Run it:

```
def func_rebind(x):  
    x = 20                # rebinds the LOCAL name  
  
p = 10  
func_rebind(p)  
print(p)                  # 10
```

⚠ SAME ANSWER, DIFFERENT REASON — THIS IS THE ONE THING TO GET RIGHT

In Java, `p` stayed 10 because `int` is a **primitive** and the number itself was copied. In Python, `p` stays 10 because `x = 20` is a **rebinding**: it points the local name `x` at a different object and leaves the object 10 untouched. Nothing was copied except an arrow. The reasoning is Java's *predict-the-output* reasoning, not Java's *primitive* reasoning.

★ THE TEST, RESTATED

Ask the same question as in class: did the function **write through** the arrow, or **rebind** it? Writing through is visible to the caller. Rebinding never is.

► NOTE

`x = 20` is a rebinding. `x[1] = 20`, `x.append(...)`, `x.sort()` are writes through the arrow. The presence of a bare `=` on the parameter name is the giveaway.

Mutable vs immutable — Python's version of the primitive/non-primitive split

Java split types by *where they live*. Python splits them by *whether they can be edited in place*:

Kind	Types	Can a function change what the caller sees?
Immutable	int, float, bool, str, tuple	No. There is no operation that edits them, so the only thing a function can do is rebind — invisible to the caller.
Mutable	list, dict, set, most objects	Yes, if it writes through the arrow. Still no, if it rebinds.

```
s = "hello"  
s = s + " world"    # NOT an edit — a NEW str object  
  
lst = [1, 2]  
lst.append(3)        # an edit — same object → [1, 2, 3]  
  
tup = (1, 2, 3)  
tup[0] = 99          # TypeError: 'tuple' object does not  
                    # support item assignment
```

★ THE MAPPING

Java's "primitives are safe to pass" becomes Python's "**immutables are safe to pass**". Different reason, same practical result — which is why the Java intuition mostly works, right up until you pass a list.

⚠ TRAP

Strings feel editable because `s += "x"` works. It is not an edit — it builds a new string every time. Doing that in a loop over 10^5 characters is $O(n^2)$. Use `"".join(parts)`.

◆ INTERVIEW

The classic bug: `def f(items=[])`. The default list is one mutable object created once, shared by every call that omits the argument — so it accumulates across calls. Use `None` as the default instead.

"Predict the output", in Python — the same answer, the same drawing

```
def fun(a):  
    a = [0] * 1          # rebinds → brand new list  
    a[0] = 100  
  
a = [10, 20, 30]  
fun(a)  
print(a[0])             # 10
```

```
def fun_mutate(a):  
    a[0] = 100           # writes THROUGH the arrow  
  
b = [10, 20, 30]  
fun_mutate(b)  
print(b[0])             # 100
```

★ ANSWERS — 10, THEN 100

Identical to Java, for identical reasons. In the first, fun got a copy of the arrow and pointed its own copy somewhere else; the caller's arrow never moved. In the second it wrote through the arrow it was given. The class diagram — main's $a \rightarrow x$, fun's $a \rightarrow y$ — is the Python diagram, unedited.

Python has **no** equivalent of C++'s `&` parameter. Like Java, rebinding a parameter can never be seen by the caller. If you want the caller to see a new object, you `return` it.

★ SAME

Every prediction you learned to make in the Java class gives the right answer in Python — provided you route it through the non-primitive case.

► DO THIS

Predict, then run: `def f(t): t = t + (4,)` called on `t = (1,2,3)`. Then the same with a list and `t.append(4)`. Explain the difference using only the words "arrow" and "rebind".

4 · Java → Python in one table

Idea	Java (from class)	Python
Call stack	One frame per call, LIFO	Identical — and a traceback prints it
Bottom frame	<code>main()</code>	<code><module></code>
Stack exhausted	<code>StackOverflowError</code> , at the real limit	<code>RecursionError</code> , at a soft limit of 1000
What's in a frame	Primitives + references	References only — there are no primitives
Where objects live	Heap (objects), stack (primitives)	Heap. All of them. Including <code>10</code> .
Seeing a reference	<code>println(ar) → [I@63947c6b</code>	<code>id(ar) / hex(id(ar))</code> — same idea
Aliasing	<code>ar2 = ar</code> → one object, two names	Identical, and <code>ar2 is ar</code> proves it
Freeing memory	Garbage collector, periodic	Reference counting (immediate) + a cycle collector
Passing an <code>int</code>	Copy of the value — caller safe	Copy of the reference — caller safe because <code>int</code> is immutable
Passing a list / array	Copy of the reference → shared	Identical
Reassign the parameter	Never visible to the caller	Never visible to the caller
The safe-to-pass types	The 8 primitives	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code> , <code>tuple</code> — the immutables

Verification. Every Python snippet in this handout is from one script executed on CPython 3.11; the outputs shown (`220 · the frame list · 1000 · RecursionError · True · [99, 0, 0] · 10 · 20 · 10 · 100 · the TypeError`) are its real output, not expected values. The `id()` values are from one run and will differ on yours — that is the point of them. Source: `notes_memory_management_python_companion_2026-09-01.py`.